ФМС — шпаргалка на защиту

Пара: сб 07.11.2026, 17:40–20:50 (практики 5–6, Телемост). **Тема 7:** «Действия после получения патента» — только пп. **7.1** (трудовой договор / ГПХ за 2 месяца с даты выдачи патента) и **7.2** (уведомление МВД после заключения договора, тоже не позднее 2 месяцев с даты выдачи). Дата отсчёта — **дата выдачи патента**, не дата ввода в систему. **Репозиторий:** `github.com/mflkee/fms-course` (публичный) · локально `~/projects/fms-course`. **Команда:** Малышева Елена (лаб. 6), Макеев Глеб (лаб. 1–5), Мальгин Олег.

---

1. Сценарий на 7–8 минут

Время	Что сказать	Что открыть
0:30	«Мы делали подсистему по теме 7: 7.1 — трудоустройство за 2 месяца с даты выдачи патента, 7.2 — уведомление МВД после договора и тоже в 2 месяца. Разделы 1–6 — предусловия, 8–11 — другие команды»	—
1:00	**Лаб. 1:** 7 прецедентов (П01–П07), 8 акторов. Фишка: **два основных исполнителя** — ИГ (7.1) и работодатель (7.2). МВД, ЕПГУ, Почта, ФНС, банк — вспомогательные	`lab1-usecases/README.md`
1:30	**Лаб. 2: 13 классов + 5 подклассов = 18**. Ядро: ИностранныйГражданин → Патент → Договор → УведомлениеМВД. 9 инвариантов ИНВ-1…9	`lab2-domain-model/README.md`
1:30	**Лаб. 3–4:** 16 системных операций, 19 SSD (7 основных + 12 альтернатив), 16 диаграмм коммуникаций (по одной на операцию)	`lab3…/operations.md`
1:00	**Лаб. 5:** BCE + слои, 46 классов: 7 экранов, 10 управляющих, 19 сущностей, 7 репозиториев, 3 шлюза. Правило «2 месяца» — в одном классе `МодульКонтроляСроков`	`lab5…/README.md`
1:30	**Лаб. 6:** FastAPI, 16/16 операций, 28 маршрутов, 84 теста, 9/9 инвариантов в `app/control/invariants.py`, слои boundary → control → entity, DI в `app/container.py`	`lab6…/README.md`
2:00	**Демо** (см. §5)	Swagger + терминал

---

2. Цифры (могут спросить дословно)

Что	Значение	Где смотреть
Прецеденты	**7** (П01–П07; П05–П07 — расширения)	`lab1-usecases/README.md`
Акторы	**8**: А1 ИГ, А2 работодатель/ГПХ, А3 ОВМ МВД, А4 ЕПГУ, А5 Почта, А6 ФНС, А7 банк, А8 админ	`lab1-usecases/actors.md`
Классы предметной области	**13 + 5 = 18** (2 абстрактных: `Договор`, `Пользователь`)	`lab2-domain-model/`
Инварианты	**9** (ИНВ-1…9)	`lab2…/README.md` §4
Системные операции	**16**	`lab3…/operations.md`
SSD	**19** = 7 основных + 12 альтернатив	`lab3…/README.md` §2
Диаграммы коммуникаций	**16** (по одной на операцию), 144 объекта, 196 сообщений	`lab4…/README.md`
Классы проектирования	**46** = 7 boundary + 10 control + 19 entity + 7 repository + 3 gateway	`lab5…/README.md` §2
HTTP-маршруты	**28** = 16 операций + 12 служебных	`lab6…/README.md`
Тесты	**84**, все проходят	`pytest`

Цепочка трассировки (одна фраза): прецедент → 16 системных операций → 16 диаграмм коммуникаций (по операции) → 46 классов (операция → класс → метод) → 28 маршрутов + 84 теста.

---

3. 9 инвариантов — по одной строке

ID	Суть
ИНВ-1	На патент — **не более одного действующего** договора
ИНВ-2	Уведомление МВД — **не раньше** даты договора и **не позднее 2 мес.** с даты выдачи патента
ИНВ-3	Договор — **не позднее 2 мес.** с даты выдачи патента
ИНВ-4	Профессия по договору **входит в перечень** профессий патента
ИНВ-5	Патент действителен только при **непрерывной** уплате авансовых НДФЛ
ИНВ-6	**Не более одного действующего** патента; срок — 12 мес.
ИНВ-7	Сумма платежа = **стоимость патента в регионе на дату платежа**
ИНВ-8	Уведомление и доказательства (квитанция/опись/трек) — **бессрочно**
ИНВ-9	Контрольные точки и напоминания **не существуют без патента** (композиция)

---

4. 16 операций → кто выполняет

№	Операция	Класс	Прецедент
1	зарегистрироватьПатент	СервисПатентов	П01
2	пометитьПатентАрхивным	СервисПатентов	П01
3	получитьСведенияОпатенте	СервисПатентов	П02
4	заключитьДоговор	СервисДоговоров	П02
5	отменитьДоговор	СервисДоговоров	П02
6	направитьУведомлениеМВД	СервисУведомлений	П03
7	зафиксироватьВозвратУведомления	СервисУведомлений	П03
8	настроитьНапоминания	СервисНапоминаний	П04
9	пересчитатьСрокиПатента	МодульКонтроляСроков	П04
10	зафиксироватьНарушениеСрока	МодульКонтроляСроков	П04
11	зарегистрироватьПлатёжНДФЛ	СервисНДФЛ	П05
12	подтвердитьПлатёж	СервисНДФЛ	П05
13	зафиксироватьПрекращениеПатентаНеуплата	СервисНДФЛ	П05
14	прекратитьДоговор	СервисДоговоров	П06
15	сформироватьРеестр	МодульОтчётности	П07
16	выгрузитьРеестр	МодульОтчётности	П07

Служебные (не операции, но есть в коде): МодульПравДоступа, ПланировщикЗадач (инициирует 9, 10, 13), ЖурналАудита (пишут все операции).

---

5. Демо — по шагам

```
cd ~/projects/fms-course/lab6-implementation
uv venv && uv pip install -r requirements.txt
.venv/bin/python -m pytest            # ожидаем: 84 passed
.venv/bin/uvicorn app.main:app --port 8099
# второй терминал:
BASE=http://127.0.0.1:8099 bash scripts/demo.sh
```

Что происходит (7 шагов): регистрация патента → трудовой договор → уведомление МВД через ЕПГУ → авансовый платёж НДФЛ → настройка напоминаний → реестр → выгрузка CSV. В конце — **повторное уведомление → 409** (это ИНВ-8, а не ошибка).

Грабли: номер патента `000777` фиксирован → на «грязном» процессе будет 409 `DUPLICATE_PATENT`; перед показом перезапустить uvicorn. Хранилища in-memory → после перезапуска состояние чистое. Порт в `demo.sh` — **8099**, в README — 8000: в Swagger открывать тот, что запущен.

Если демо не идёт — открыть `/docs` и прогнать 2–3 запроса руками (регистрация патента, заключение договора, уведомление → 409).

---

6. Вопросы — короткие ответы (по одному предложению)

1. **Почему только 7.1 и 7.2?** Тема распределена по командам; остальные разделы — предусловия или чужие темы.
2. **Почему МВД вспомогательный?** В теме 7 система уведомление готовит и отправляет, а решения по патенту принимает МВД.
3. **Почему 7, а не 2 прецедента?** 7.1 и 7.2 — разные обязанности разных исполнителей; плюс расширения (НДФЛ, прекращение договора, отчётность).
4. **От какой даты считаются 2 месяца?** От даты выдачи патента (ИНВ-2/3); поздний ввод в систему срок не сдвигает.
5. **Чем SSD отличается от диаграммы коммуникаций?** SSD — «чёрный ящик»: актор и одна линия системы; коммуникации — внутренние объекты и сообщения.
6. **Почему 19 SSD при 16 операциях?** Одна операция может встречаться в нескольких сценариях; альтернативы показывают ветви, а не новые операции.
7. **Почему 16 операций, а ориентир 10–14?** Альтернативы потребовали отдельных операций (архивирование, отмена, возврат, подтверждение платежа, прекращение из-за неуплаты).
8. **Где живёт правило «2 месяца»?** Один класс МодульКонтроляСроков + Clock в коде; сроки считаются от даты выдачи.
9. **Что такое BCE?** Boundary (экран) → Control (сценарий/инварианты) → Entity (понятие предметной области), вниз — репозитории и шлюзы.
10. **Почему классы без типов языка?** Требование лаб. 5: предметные типы (Дата, Сумма, Идентификатор), ни String, ни int.

11. **Как проверяются инварианты?** `app/control/invariants.py`, вызов из сервисов (не из роутеров), на каждый инвариант — тест.
12. **Почему инварианты в сервисах?** Роутер можно обойти, сервис — нет; тесты бьют прямо в сервисы.
13. **Почему 46 классов, а сущностей меньше?** 19 сущностей + 4 вспомогательные структуры; остальное — экраны, управляющие, репозитории, шлюзы.
14. **Что заглушки?** ЕПГУ, Почта, ФНС — детерминированные ответы + режим отказа; подписи не проверяются.
15. **Аутентификация?** Учебная: логин в заголовке `X-User-Login` (по умолчанию `admin`), без пароля и ЕСИА.
16. **Хранение?** In-memory за интерфейсами репозиториев; без `delete` — так реализован ИНВ-8.
17. **Деньги?** `Decimal`, в JSON — строкой `"6000.00"`, чтобы не терять копейки.
18. **Как тестируются сроки?** `Clock`/`FixedClock`, без `freezegun` и «сегодня» в коде.
19. **Идемпотентность?** Повторный платёж за тот же период с той же суммой ⟶ 200 и тот же платёж, без дубля.
20. **Права?** `МодульПравДоступа`: `auditor` формирует отчёт, но выгрузка принудительно обезличена (`anonymize=false` ⟶ 403 + аудит).
21. **Что не реализовано?** PostgreSQL/ORM, реальные шлюзы, фоновый планировщик, хранение файлов, форматная валидация ИНН/серии.
22. **Что дальше?** PostgreSQL + Alembic (инварианты закрепить индексами), реальные ЕПГУ/СМЭВ/Почта/ФНС, APScheduler/Celery.
23. **Кто что делал?** Лаб. 1–5 — Макеев, лаб. 6 — Малышева; на защите уметь провести трассировку по обеим частям.
24. **Как трассируется операция?** Таблица 3.3 в README лаб. 6 и `operations-mapping.md` лаб. 5: операция ⟶ класс ⟶ метод ⟶ эндпоинт.
25. **Почему FastAPI?** Pydantic-валидация, Swagger для демо, скорость; модель от фреймворка не зависит.

---

7. Техчек перед парой

- ☐ Телемост: звук, экран, уведомления выключены.
- ☐ Вкладки: GitHub `fms-course`, `lab6-implementation/README.md`, Swagger `127.0.0.1:8099/docs`, терминал с uvicorn.
- ☐ Демо прогрето на чистом процессе; `demo.sh` до показа не запускать.
- ☐ Под рукой: этот лист (§2 цифры, §6 вопросы) + полная подготовка.
- ☐ Если Елены нет — провести лаб. 6 самому (структура в §1, ограничения в §6 п. 21).

Полная версия со всеми диаграммами, контрактами операций и развёрнутыми ответами — ФМС — подготовка к защите.

Для заметок